Distributed LLM Inference on the Edge

Running Llama-3.2-3B on a cluster of Raspberry Pis

Yuzhou Zhang Yunfan Zhang Shuo Zhang Wenqi Zhao

DAT300/DIT616 – Group 7
Supervisors: Yixing Zhang, Romaric Duvignau

Final presentation, October 2026

Outline

1. Background
2. Results and bottlenecks
3. Conclusion

Why run an LLM on edge devices?

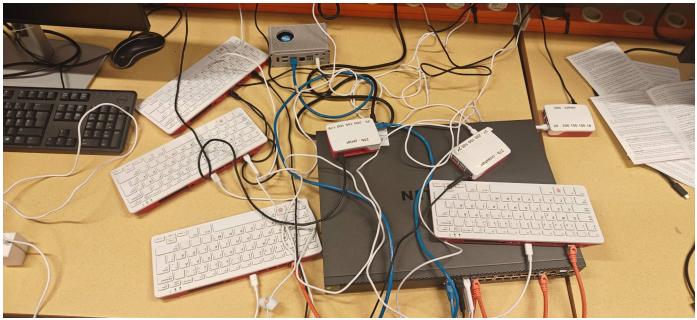
- Cyber-physical systems (robots, cars, smart homes, factories) increasingly want language models close to the sensors
- Cloud LLMs send the data away, add latency and need a connection
- One Raspberry Pi can run a small model, but slowly
- Can several cheap boards work together and be faster?

Research questions

1. How does speed scale with the number of nodes?
2. Pipeline or tensor parallelism?
3. What is the bottleneck?
4. Can we fix it without losing accuracy?

Model: Llama-3.2-3B-Instruct, 4-bit (1.9 GB)

Our cluster

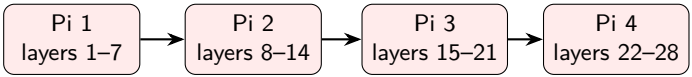


- 7 Raspberry Pis (Pi 400 and Pi 4B)
- 1 LubanCat-5 board (RK3588)
- One Gigabit switch

	Pi	RK3588
Mem. bandwidth	4 GB/s	22 GB/s
Decode, alone	1.84 tok/s	12.28 tok/s

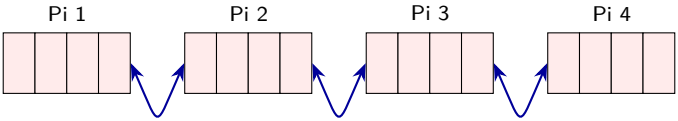
The problem: how do you split a model over devices?

Pipeline parallel (PP)



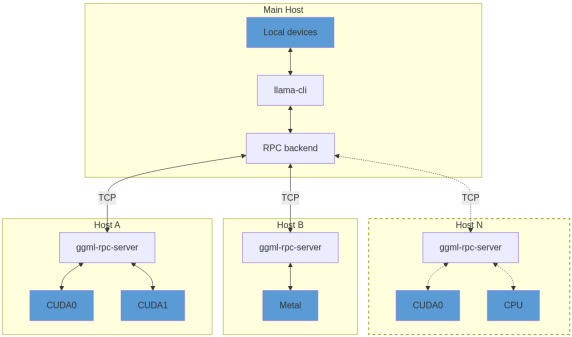
the token visits one device after another: only one device works at a time

Tensor parallel (TP)



every layer is split over all devices: all work at once, but they must sum their results (an *allreduce*) twice per layer, 56 times per token

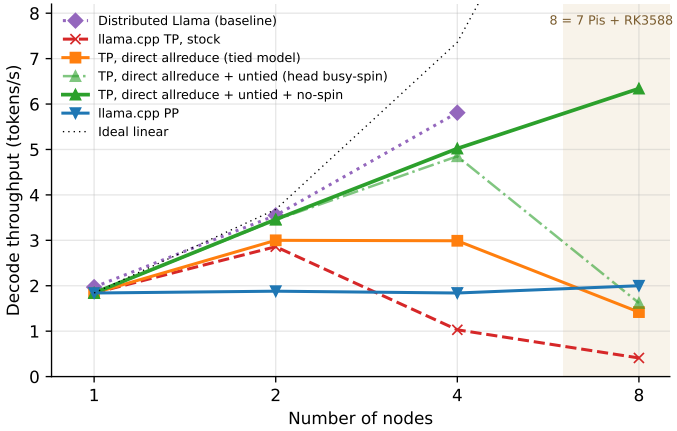
Our approach



llama.cpp RPC backend (from the official docs)

- Baseline: Distributed Llama, 1/2/4 nodes
- Then llama.cpp RPC with 1–8 devices
  - pipeline parallel (PP): split by layers
  - tensor parallel (TP): split every layer
- To find bottlenecks we logged
  - CPU, memory and network on every device
  - time spent per RPC command

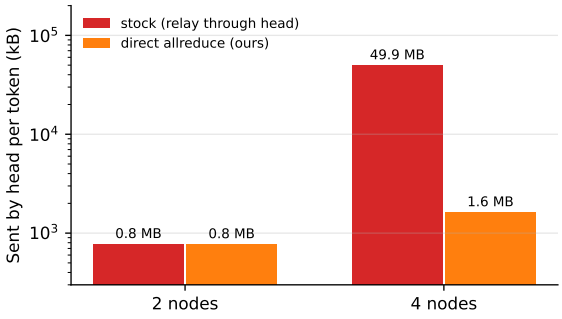
Decode speed from 1 to 8 nodes



- PP is never faster than one Pi
- Stock TP gets *slower* with more nodes (0.41 tok/s on 8)
- After our changes: 6.34 tok/s on 8 nodes, 3.4× one Pi

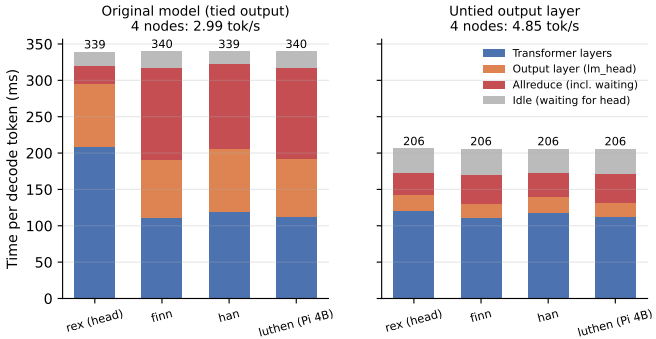
Three bottlenecks explain the gap.

Bottleneck 1: all traffic goes through the head



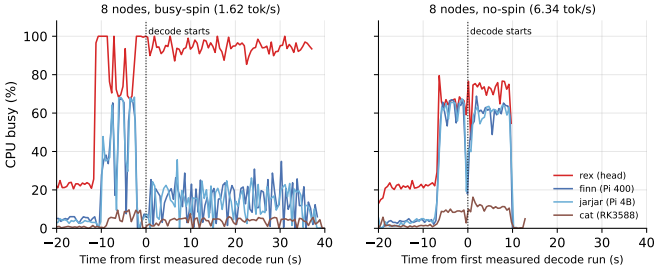
- With more than 2 nodes, the head collects and redistributes every partial sum
- On 4 nodes the head sends about 50 MB per token
- We added a direct allreduce between the nodes ($\log_2 N$ steps)
- 4 nodes: 1.03 to 2.99 tok/s

Bottleneck 2: the output layer is not split



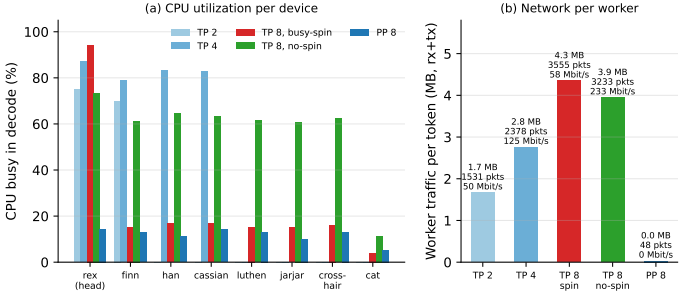
- The timing log showed 80–85 ms per token on the output layer on *every* node
- Llama-3.2 shares it with the input embedding, so llama.cpp copies it instead of splitting it
- We saved a separate copy in the model file (same output, same perplexity)
- 4 nodes: 2.99 to 4.85 tok/s

Bottleneck 3: the head's CPU at 8 nodes



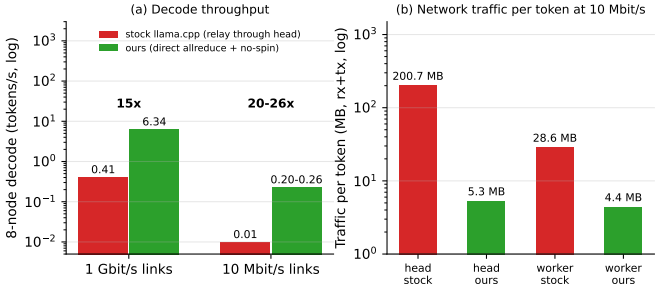
- Head CPU 94%, workers only 15%
- Workers waited 470 of 631 ms per token
- With gdb we found 8 threads busy-waiting on the head's 4 cores
- We added an option to let them sleep instead
- 8 nodes: 1.62 to 6.34 tok/s

Is the network the bottleneck?



- On Gigabit: no. Each worker uses at most 233 of 940 Mbit/s
- Fixing bottleneck 3 made it 3.9× faster with the *same* traffic
- One Pi alone is limited by memory bandwidth

What if the network is slow? (10 Mbit/s)



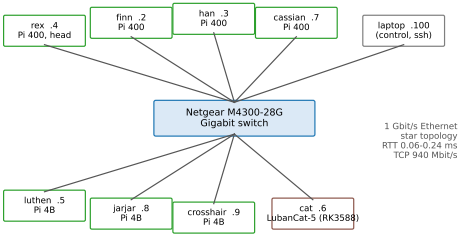
- We limited all 8 devices to 10 Mbit/s
- Stock: 200 MB per token through the head, 0.01 tok/s
- Ours: 5.3 MB, 0.20–0.26 tok/s
- 20–26× faster here vs. 15× on Gigabit

Demo: chatting with the 8-node cluster

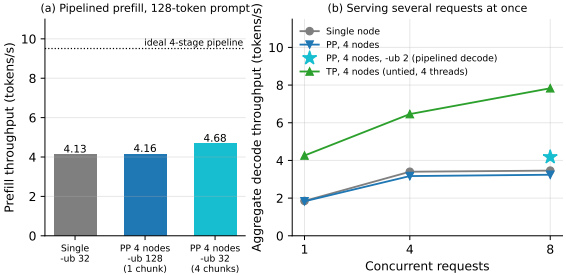
On the head node

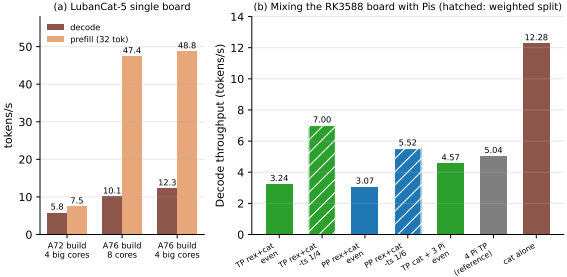
```
$ scripts/chat.sh "What is a Raspberry Pi?"
```

- Tensor parallel over all 8 devices
- With our changes: about 6 tokens/s
- Stock llama.cpp: one token every 2.4 s



Pipeline parallelism and a faster board





- PP only helps with several requests at once (+29%)
- Its real use: models that don't fit on one board

- Compiling for the RK3588's CPU: 5.84 to 12.28 tok/s
- Giving it a bigger share helps (TP on 2 nodes: 3.24 to 7.00)
- But it alone beats the whole Pi cluster for this model

Practical challenges in the lab

- The switch lost its DHCP settings after a reset, so we gave every board a static IP
- One Pi rebooted mid-test because of a weak power supply
- The Pis have no real-time clock, so we corrected clock offsets before comparing logs
- Our first monitoring script (shell) slowed decoding by 5%; a small C program fixed this
- Uneven splits on 8 nodes fall back to the slow path, so we left them out

Conclusions and insights

What we found

1. TP scales to 3.4× on 8 Pis; PP does not speed up a single request
2. TP needs far more communication than PP
3. The limits were the head relay, the unsplit output layer and the head's CPU, not network bandwidth
4. Three small changes, no accuracy loss

New challenges that came up

- Each fix revealed the next bottleneck (8 nodes: the head's CPU)
- Mixed devices need uneven splits, which the fast allreduce does not support yet
- One modern board beats 7 Pis for a 3B model: clusters pay off for models that don't fit on one board

Thank you! Questions?

Code and data: https://github.com/44sunsetsf/dat300-edge-llm